

Secure Network Application Development Using TCP

NAME: KAUSHAL KUMAR

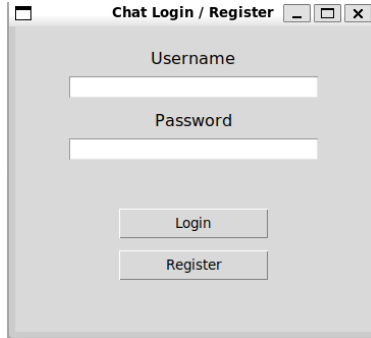
ROLL NR: CSB24069

1. Objective

Enhance the GUI-based multi-client TCP application by implementing practical security mechanisms including authentication, password protection, session management and input validation. The emphasis is on application security rather than advanced cryptography.

2. Security Features Implemented

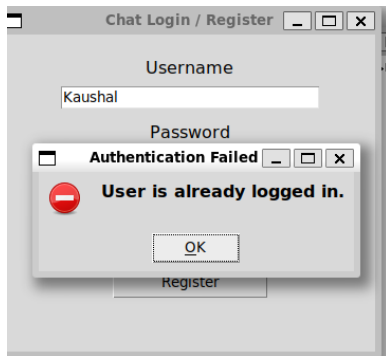
- **User Authentication:** Implemented a username and password based authentication system.



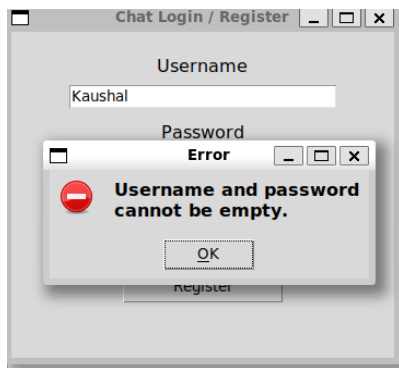
- **Secure Password Storage:** Passwords are stored securely using SHA-256 hashes instead of plaintext.

	A	B	C	D	E	F	G	H
1	username	password_hash						
2	Kaushal	40510175845988f13f6162ed8526f0b09f73384467fa855e1e79b44a56562a58						
3	kaushal	40510175845988f13f6162ed8526f0b09f73384467fa855e1e79b44a56562a58						
4	Alice	b9677bd7630cca66d36ea2d10cd0eaa4002dee7afeaabb88c4e3c6fa751edf7a						
5	Bob	d1cdb85a36c673b8585c895c9c43fa8d967ee5c7166af737af6e04d351d525f2						
6								

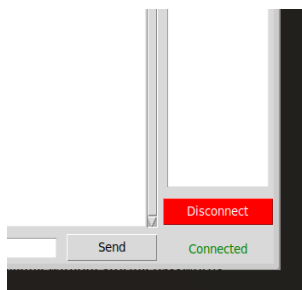
- **Duplicate Login Prevention:** The application actively tracks active sessions to prevent the same user from logging in simultaneously.



- **Input Validation:** The system strictly rejects invalid usernames, empty passwords, oversized messages, and unsupported commands.



- **Failed Login Protection:** To prevent brute-force attacks, logins are temporarily blocked after five consecutive failed attempts.
- **Session Management & Secure Logging:** The server supports manual logout, enforces an inactivity timeout for idle sessions, and maintains secure logging without ever storing passwords.



3. System Architecture

The application follows a centralized Client-Server architecture built over the TCP/IP protocol. It is designed to be deployed and tested on a Mininet simulated network utilizing a single switch and five hosts (topo single,5).

The architecture is divided into three primary components:

- Server Component (server_multi.py):
 - Acts as the central node, managing all incoming TCP socket connections.
 - Utilizes a multi-threaded design, spawning a new daemon thread for every authenticated client to handle concurrent communications without blocking.
 - Maintains global state variables (protected by threading.Lock) to track active sessions, connected IPs, and failed login attempts in memory.
 - Interfaces with local file storage to persistently manage data, including users.csv for hashed credentials, chat_history.csv for message history, and security_log.txt for audit trails.
- Client Component (client_gui.py):
 - Features a graphical user interface (GUI) developed using Python's tkinter library, separating networking logic from the UI presentation.
 - Operates using two primary threads: the main thread handles user input and GUI updates, while a background daemon thread continuously listens for incoming broadcasts and private messages from the server.
- Communication Protocol:
 - Data transmission relies on standard TCP sockets (AF_INET, SOCK_STREAM).
 - The application uses a custom delimiter-based payload structure for the authentication phase (e.g., ACTION|username|password) and specific string prefixes (e.g., /msg, /quit) to route commands.

4. Implementation

The application is implemented in Python, extending the existing TCP socket foundation with focused security enhancements. The core implementation details include:

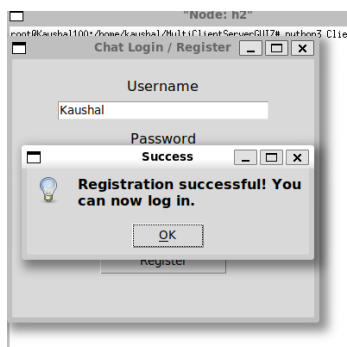
- GUI and Networking: The client application uses `tkinter` for the graphical interface, maintaining a strict separation between the visual elements and the background `socket` and `threading` network logic.
- Authentication & Storage: The server manages a user database using a `users.csv` file. Passwords are never stored or transmitted in plain text after processing; they are immediately hashed using Python's `hashlib.sha256()` module.
- State Management: The server uses in-memory dictionaries protected by thread locks to track active user sessions (preventing duplicate logins) and monitor connection IPs.
- Input Validation: Both client and server scripts validate inputs using Regular Expressions (Regex) for usernames, enforce length requirements for passwords, and restrict incoming socket data payloads to a maximum of 1024 bytes to prevent buffer issues.

- Secure Logging: A custom logging function writes critical network and authentication events to `security_log.txt` with timestamps, strictly avoiding the exposure of user passwords in the logs or console.

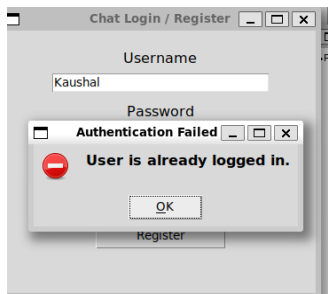
5. Testing

Testing was conducted using a simulated network environment via Mininet with a single-switch, five-host topology (`sudo mn --topo single,5`) to establish and verify network connectivity. The following scenarios were systematically tested to ensure all security mechanisms functioned as intended:

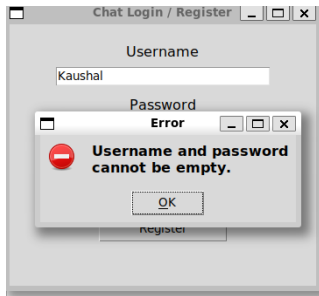
- Authentication & Registration: Verified that new users can register successfully and that logins are only granted with the correct, matching credentials.



- Duplicate Login Prevention: Attempted to log in using an already active username from a different host, confirming that the server successfully identified and rejected the secondary connection.



- Input Validation: Tested various edge cases by submitting empty passwords, invalid username characters, unsupported commands, and oversized messages, ensuring the system rejected them gracefully without crashing.



- Session Management: Left an authenticated client idle to confirm that the server correctly tracked inactivity and automatically terminated the connection after the predefined timeout period.
- Secure Logging: Reviewed the generated security_log.txt file to ensure all critical events (logins, failures, disconnects) were accurately recorded and that no plaintext passwords were leaked in the logs.

6. Wireshark Verification

Network traffic was captured and analyzed using Wireshark within the Mininet environment to verify the security and behavior of the application's communication. The capture was filtered using `tcp.port == 5000` to isolate the chat application's traffic.

- Login and Authentication: Captured the initial TCP 3-way handshake and the transmission of the authentication payload. While the payload itself is transmitted over standard TCP, the server logs confirm the secure processing of the hashed credentials.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	10.0.0.5	10.0.0.1	TCP	74	34638 → 5000 [SYN] Seq=0 Win=42340 Len=0 MSS=1460 SACK_PERM TSval=3610325504 TSecr=0 WS=612
2	0.000084754	10.0.0.1	10.0.0.5	TCP	74	5000 → 34638 [SYN, ACK] Seq=0 Ack=1 Win=43440 Len=0 MSS=1460 SACK_PERM TSval=3606999433 TSecr=3610325504 WS=512
3	0.001721207	10.0.0.5	10.0.0.1	TCP	66	34638 → 5000 [ACK] Seq=1 Ack=1 Win=42496 Len=0 TSval=3610325507 TSecr=3606999433

- Failed Login: Captured the traffic corresponding to incorrect password submissions and the server's subsequent rejection packets, verifying the network-level response to brute-force attempts.

16	15.498124590	10.0.0.1	10.0.0.2	TCP	66	5000 → 53594 [FIN, ACK] Seq=37 Ack=17 Win=43520 Len=0
17	15.539988883	10.0.0.2	10.0.0.1	TCP	66	53594 → 5000 [ACK] Seq=17 Ack=38 Win=42496 Len=0
18	20.736120300	5e:61:dc:a8:3e:b8	fa:a5:91:8d:4a:39	ARP	42	Who has 10.0.0.2? Tell 10.0.0.1
19	20.736493117	fa:a5:91:8d:4a:39	5e:61:dc:a8:3e:b8	ARP	42	10.0.0.2 is at fa:a5:91:8d:4a:39

- Authenticated Communication: Monitored the exchange of broadcast and private messages between clients, confirming that message payloads remain strictly within the predefined size limits established during input validation.

```
LOGIN|kaushal|1000SUCCESS|Authenticated.
##USERS##:kaushal
--- Your last 5 messages ---
[2026-07-15 14:32:23] hellow
[2026-07-15 14:32:38] Big surprise for you
-----
Hello everyone[OK]
once again[OK]
I'm here[OK]
/quit
```

- Logout and Session Termination: Captured the explicit `/quit` command transmission and the resulting TCP connection termination (FIN/ACK packets), as well as automated disconnections triggered by the 5-minute inactivity timeout.

57	119.904861675	10.0.0.2	10.0.0.1	TCP	66	35730 → 5000 [ACK] Seq=51 Ack=187 Win=42496 Len=0 TSval=2415087097 TSecr=2133901886
58	126.637329058	10.0.0.2	10.0.0.1	TCP	71	35730 → 5000 [PSH, ACK] Seq=51 Ack=187 Win=42496 Len=5 TSval=2415093830 TSecr=2133901886
59	126.638412284	10.0.0.1	10.0.0.2	TCP	66	5000 → 35730 [FIN, ACK] Seq=187 Ack=56 Win=43520 Len=0 TSval=2133908620 TSecr=2415093830
60	126.638576584	10.0.0.2	10.0.0.1	TCP	66	35730 → 5000 [FIN, ACK] Seq=56 Ack=188 Win=42496 Len=0 TSval=2415093831 TSecr=2133908620
61	126.638613252	10.0.0.1	10.0.0.2	TCP	66	5000 → 35730 [ACK] Seq=188 Ack=57 Win=43520 Len=0 TSval=2133908620 TSecr=2415093831

7. Conclusion

This assignment successfully enhanced a standard GUI-based multi-client TCP application by integrating critical application-layer security mechanisms. By moving beyond basic networking and implementing features such as SHA-256 password hashing, strict input validation, and session timeouts, the application's resilience against unauthorized access, brute-force attempts, and malicious payloads was significantly improved.

The deployment of the application within the Mininet simulated network and the subsequent traffic analysis using Wireshark provided practical, real-world verification of these security features in action. Ultimately, this project reinforced core concepts in secure network development, demonstrating that robust server-side logic, secure credential management, and active connection tracking are fundamental to maintaining a secure and reliable client-server environment.